

Source Link: https://github.com/Bharathp953/Azure/blob/main/log_reg.ipynb

```
\documentclass[a4paper,12pt]{article}
```

```
\usepackage{graphicx}
```

```
\usepackage{booktabs}
```

```
\usepackage{amsmath}
```

```
\usepackage{hyperref}
```

```
\usepackage{geometry}
```

```
\geometry{margin=1in}
```

```
\title{Logistic Regression on Fashion-MNIST: Experimental Analysis}
```

```
\author{}
```

```
\date{}
```

```
\begin{document}
```

```
\maketitle
```

```
\section{Matplotlib Plots}
```

Two visualizations are included in this report:

```
\begin{itemize}
```

```
\item \textbf{Predicted Images (from plot\_images):}
```

Displays a 3×3 grid of sample test images with their true and predicted labels.

```
\item \textbf{Weight Visualizations (from plot\_weights):}
```

Shows 10 heatmaps (one per class) representing learned weight patterns.

```
\end{itemize}
```

```
\section{Optimizer Comparison}
```

\textbf{Optimizers Used:}

\begin{itemize}

\item \texttt{tf.optimizers.SGD(learning_rate=0.01)}

\item \texttt{tf.optimizers.RMSprop(learning_rate=0.001)}

\item \texttt{tf.optimizers.Adagrad(learning_rate=0.01)}

\item \texttt{tf.optimizers.Adam(learning_rate=0.001)}

\end{itemize}

\textbf{Optimizer Comparison Summary:}

\begin{tabular}{@{}lccc@{}}

\toprule

**Optimizer & Convergence Speed & Generalization & Notes **

\midrule

**SGD & Slow & Decent & Manual learning rate tuning required **

**RMSProp & Moderate & Good & Handles noisy gradients well **

**Adagrad & Slower & Stable & Useful for sparse features **

**Adam & Fastest & Excellent & Reaches superior minima in fewer epochs **

\bottomrule

\end{tabular}

\bigskip

\textbf{Conclusion:}

The Adam optimizer converges fastest and achieves the highest validation/test accuracy for this linear model.

\section{Training for Longer Epochs}

\begin{itemize}

\item \textbf{Increased:} $n_{\text{epochs}} = 50$ (instead of 15)

\item \textbf{Observation:} Training loss decreases gradually.

Validation accuracy plateaus around 84–86% after ~ 20 epochs.

Beyond this point, training accuracy increases while validation stagnates, indicating overfitting.

\item \textbf{Summary:} There is an optimal training duration; further epochs do not improve performance.

\end{itemize}

\section{Changing Train/Validation Split}

\begin{itemize}

\item \textbf{55k train / 5k validation:}

Train Acc = 0.90, Val Acc = 0.85 — Balanced performance.

\item \textbf{50k train / 10k validation:}

Train Acc = 0.89, Val Acc = 0.86 — Slight improvement in validation accuracy.

\item \textbf{58k train / 2k validation:}

Train Acc = 0.91, Val Acc = 0.83 — Validation less reliable due to small set. **\end{itemize}**

\textbf{Conclusion:}

Keeping approximately 10% of the data for validation provides a more stable and reliable evaluation.

\section{Train/Validation Accuracy Over Time}

\begin{itemize}

\item Both accuracies increase initially.

\item After about 20 epochs, training accuracy continues to rise while validation flattens or slightly decreases.

\item \textbf{Interpretation:} Mild overfitting observed — typical for simple linear models.

\end{itemize}

\section{Effect of Batch Size}

\begin{tabular}{@{}lccc@{}}

\toprule

Batch Size & Time per Epoch & Final Val Accuracy & Notes \\

\midrule		
32	& Slow	& 0.86 & Noisy updates, better generalization \\\
64	& Moderate	& 0.86 & Balanced speed and generalization \\\
128	& Fast	& 0.85 & Default batch size performs well \\\
256	& Faster	& 0.84 & Slight drop in accuracy \\\
512	& Fastest	& 0.82 & Weak minima, mild overfitting \\\
\bottomrule		
\end{tabular}		

\bigskip

\textbf{Conclusion:}

Smaller batches generalize better but are slower; larger batches converge faster but may overfit slightly.

\section{GPU vs CPU Timing}

\begin{tabular}{@{}lcc@{}}

\toprule

Device & Time/Epoch & Comment \\\

\midrule

CPU & $\sim 3\text{--}5$ sec & Adequate for small models \\\

GPU & $\sim 1\text{--}2$ sec & Slightly faster; shines on larger networks \\\

\bottomrule

\end{tabular}

\bigskip

\textbf{Conclusion:}

For simple logistic regression, CPU performance is sufficient; GPUs are advantageous for deeper architectures.

\section{Overfitting Analysis}

\textbf{Observation:}

\begin{itemize}

\item After $\sim 25-30$ epochs: Train Acc $\approx 0.90+$, Val Acc ≈ 0.85 .

\item Small but consistent overfitting.

\end{itemize}

\textbf{Causes:}

\begin{itemize}

\item Model memorizes frequent patterns.

\item Lack of regularization (no L2 or dropout).

\item No data augmentation.

\end{itemize}

\textbf{Remedies:}

\begin{itemize}

\item Early stopping (stop if val accuracy doesn't improve for 5 epochs).

\item L2 regularization:

\[

loss = loss_fn(logits, y_batch Oh) + 0.001 \times tf.nn.l2_loss(w)

\]

\end{itemize}

\textbf{Conclusion:}

Overfitting is mild and manageable with early stopping or L2 regularization.

\section{Comparison with Random Forest and SVM}

\begin{tabular}{@{}lccc@{}}

\toprule

Model & Test Accuracy & Training Time & Notes \\\

\midrule

Logistic Regression (TF) & 85\% & Fast & Baseline model \\\

Random Forest & $\sim 80\%$ & Slow & Struggles with high-dimensional input \\\

Linear SVM & $\sim 86\%$ & Moderate & Comparable to logistic regression \\\

RBF SVM & $\sim 88\%$ & Very Slow & Best accuracy but not scalable \\

\\bottomrule

\\end{tabular}

\\bigskip

\\textbf{Conclusion:}

Even a linear deep learning model performs well.

SVMs can match or slightly exceed performance but are computationally heavier.

\\section{Weight Clustering (t-SNE / K-Means)}

\\begin{itemize}

\\item t-SNE visualized 784-D weight vectors (one per class) into 2D space.

Each class forms a distinct cluster.

\\item Semantically similar classes (e.g., ``T-shirt" and ``Shirt") appear closer together.

\\item K-Means (k=10) centers correspond roughly to class prototypes.

\\item \\textbf{Conclusion:} Learned weights exhibit meaningful structure and separable clusters.

\\end{itemize}

\\section{Summary of Findings}

\\begin{itemize}

\\item \\textbf{Optimizer:} Adam generalized best and converged fastest.

\\textit{Conclusion:} Adam best overall.

\\item \\textbf{Epochs:} More epochs improve training but risk overfitting.

\\textit{Conclusion:} Use early stopping.

\\item \\textbf{Train/Validation Split:} Balanced split yields stable validation.

\\textit{Conclusion:} 55k/5k recommended.

\\item \\textbf{Batch Size:} Smaller batches generalize better.

\\textit{Conclusion:} Trade-off between speed and performance.

\\item \\textbf{Device:} GPU slightly faster.

\\textit{Conclusion:} CPU sufficient for small models.

\\item \\textbf{Overfitting:} Mild after 20+ epochs.

\textit{Fix:} L2 regularization or early stopping.

\item \textbf{Random Forest / SVM:} Similar or slightly better accuracy but slower.

\textit{Conclusion:} TF logistic regression more efficient.

\item \textbf{Weight Clusters:} Clear separation of class prototypes.

\textit{Conclusion:} Model learned meaningful features.

\end{itemize}

\end{document}